

```

1  import flet as ft
2  import paho.mqtt.client as mqtt
3  import random
4  import datetime
5  import threading
6  import time
7  import json
8  import os
9  import tempfile # Garante um diretório com permissão de escrita em qualquer SO
10
11  # --- CONFIGURAÇÕES MQTT ---
12  MQTT_SERVER = "emqx.ifspb-czemnumeros.com.br"
13  MQTT_PORT = 1883
14  MQTT_USER = "esp12_01"
15  MQTT_PASS = "esp12_01"
16
17  # Dicionário global para controlar o estado das automações
18  automacoes = {
19      "casa/rele1": {"timer_minutos": 0, "hora_agenda": "18:00", "dias_agenda": set(),
20      "timer_objeto": None},
21      "casa/rele2": {"timer_minutos": 0, "hora_agenda": "18:00", "dias_agenda": set(),
22      "timer_objeto": None}
23  }
24
25  # --- DEFINIÇÃO DO CAMINHO COMPATÍVEL MULTIPLATAFORMA ---
26  # tempfile.gettempdir() aponta para uma pasta com permissões totais tanto no PC
27  # quanto no Android/iOS
28  CONFIG_FILE = os.path.join(tempfile.gettempdir(), "config_automacoes_jardim.json")
29  print(f"DEBUG: Caminho seguro utilizado = {CONFIG_FILE}")
30
31  def main(page: ft.Page):
32      page.title = "Meu Jardim - Automação"
33      page.theme_mode = ft.ThemeMode.DARK
34      page.window_width = 400
35      page.window_height = 700
36      page.horizontal_alignment = "center"
37
38      # --- BARRA SUPERIOR ---
39      page.appbar = ft.AppBar(
40          title=ft.Text("🏠 AUTOMAÇÃO", size=22, weight="bold"),
41          center_title=True,
42          bgcolor="surface",
43      )
44
45      # --- STATUS DO BROKER ---
46      status_led = ft.Container(width=12, height=12, bgcolor="red", border_radius=6)
47      status_text = ft.Text("Desconectado", color="red", weight="bold")
48
49      # --- CLIENTE MQTT ---
50      client_id = f'flet-mqtt-{random.randint(0, 1000)}'
51      mqttc = mqtt.Client(client_id=client_id)
52      mqttc.username_pw_set(MQTT_USER, MQTT_PASS)
53
54      def on_connect(client, userdata, flags, rc):
55          if rc == 0:
56              status_led.bgcolor = "green"
57              status_text.value = "Online"
58              status_text.color = "green"
59          else:
60              status_led.bgcolor = "red"
61              status_text.value = f"Erro {rc}"
62              status_text.color = "red"
63          page.update()
64
65      mqttc.on_connect = on_connect
66
67      # --- FUNÇÃO ATUADORA (MUDAR RELE) ---
68      def mudar_rele(e):
69          topic = e.control.data
70          msg = "ON" if e.control.value else "OFF"
71
72          if msg == "OFF" and automacoes[topic]["timer_objeto"] is not None:
73              try:

```

```

71         automacoes[topic]["timer_objeto"].cancel()
72         automacoes[topic]["timer_objeto"] = None
73     except Exception:
74         pass
75
76     try:
77         mqttc.publish(topic, msg)
78     except Exception as err:
79         print(f"Erro MQTT: {err}")
80
81 # --- PERSISTÊNCIA LOCAL ---
82 def salvar_dados_no_disco():
83     try:
84         dados_para_salvar = {
85             "casa/rele1": {
86                 "timer_minutos": automacoes["casa/rele1"]["timer_minutos"],
87                 "hora_agenda": automacoes["casa/rele1"]["hora_agenda"],
88                 "dias_agenda": list(automacoes["casa/rele1"]["dias_agenda"])
89             },
90             "casa/rele2": {
91                 "timer_minutos": automacoes["casa/rele2"]["timer_minutos"],
92                 "hora_agenda": automacoes["casa/rele2"]["hora_agenda"],
93                 "dias_agenda": list(automacoes["casa/rele2"]["dias_agenda"])
94             }
95         }
96         with open(CONFIG_FILE, "w") as f:
97             json.dump(dados_para_salvar, f)
98         print("DEBUG: Dados guardados em segurança!")
99     except Exception as e:
100         print(f"Erro ao salvar dados: {e}")
101
102 # --- LÓGICA DO TEMPORIZADOR (TIMER) ---
103 def desligar_por_timeout(pino_rele, switch_componente):
104     try:
105         mqttc.publish(pino_rele, "OFF")
106         switch_componente.value = False
107         automacoes[pino_rele]["timer_objeto"] = None
108         automacoes[pino_rele]["timer_minutos"] = 0
109
110         page.snack_bar = ft.SnackBar(ft.Text(f"🔌 {pino_rele} desligado! Tempo terminou.))
111         page.snack_bar.open = True
112         page.update()
113     except Exception as e:
114         print(f"Erro no timeout do timer: {e}")
115
116 def acionar_temporizador(pino_rele, txt_timer, switch_componente):
117     try:
118         minutos = int(txt_timer.value)
119     except ValueError:
120         page.snack_bar = ft.SnackBar(ft.Text("Insira um número válido de minutos!"))
121         page.snack_bar.open = True
122         page.update()
123         return
124
125     if minutos <= 0:
126         page.snack_bar = ft.SnackBar(ft.Text("O tempo deve ser maior que zero!"))
127         page.snack_bar.open = True
128         page.update()
129         return
130
131     if automacoes[pino_rele]["timer_objeto"] is not None:
132         automacoes[pino_rele]["timer_objeto"].cancel()
133
134     mqttc.publish(pino_rele, "ON")
135     switch_componente.value = True
136
137     page.snack_bar = ft.SnackBar(ft.Text(f"🕒 Temporizador iniciado por {minutos} minuto(s)"))
138     page.snack_bar.open = True
139     page.update()
140

```

```

141     segundos = minutos * 60
142     t = threading.Timer(segundos, desligar_por_timeout, args=[pino_rele,
143         switch_componente])
144     automacoes[pino_rele]["timer_objeto"] = t
145     automacoes[pino_rele]["timer_minutos"] = minutos
146     t.start()
147
148     salvar_dados_no_disco()
149
150     # --- INPUTS VISUAIS ---
151     txt_timer_d6 = ft.TextField(label="Minutos", value="30", width=90,
152         text_align="center", dense=True)
153     txt_hora_d6 = ft.TextField(label="HH:MM", value="18:00", width=90,
154         text_align="center", dense=True)
155     buttons_d6 = {}
156
157     txt_timer_d7 = ft.TextField(label="Minutos", value="30", width=90,
158         text_align="center", dense=True)
159     txt_hora_d7 = ft.TextField(label="HH:MM", value="18:00", width=90,
160         text_align="center", dense=True)
161     buttons_d7 = {}
162
163     switch_luz = ft.Switch(value=False, data="casa/rele1", on_change=mudar_rele)
164     switch_tomada = ft.Switch(value=False, data="casa/rele2", on_change=mudar_rele)
165
166     # --- CARREGA DADOS DO ARQUIVO ANTES DE CRIAR INTERFACE ---
167     def carregar_dados_local():
168         try:
169             if os.path.exists(CONFIG_FILE):
170                 with open(CONFIG_FILE, "r") as f:
171                     dados_dict = json.load(f)
172                     print(f"DEBUG: Dados carregados: {dados_dict}")
173
174                     for pino in ["casa/rele1", "casa/rele2"]:
175                         if pino in dados_dict:
176                             automacoes[pino]["hora_agenda"] =
177                                 dados_dict[pino].get("hora_agenda", "18:00")
178                             automacoes[pino]["timer_minutos"] =
179                                 dados_dict[pino].get("timer_minutos", 0)
180                             dias_carregados = dados_dict[pino].get("dias_agenda", [])
181                             automacoes[pino]["dias_agenda"] = set(dias_carregados)
182         except Exception as e:
183             print(f"Erro ao carregar dados: {e}")
184
185     carregar_dados_local()
186
187     # --- CONSTRUTOR DE BLOCOS ---
188     # --- CONSTRUTOR DE BLOCOS ---
189     def criar_bloco_automacao(pino_rele, txt_timer, txt_hora, buttons_dict,
190         switch_componente):
191         def salvar_agenda(e):
192             automacoes[pino_rele]["hora_agenda"] = txt_hora.value
193             salvar_dados_no_disco()
194             page.snack_bar = ft.SnackBar(ft.Text(f"Agenda de {pino_rele} salva para as
195                 {txt_hora.value}!"))
196             page.snack_bar.open = True
197             page.update()
198
199         def alternar_dia_btn(e):
200             container_alvo = e.control.content
201             dia = container_alvo.data
202             texto_alvo = container_alvo.content
203
204             if dia in automacoes[pino_rele]["dias_agenda"]:
205                 automacoes[pino_rele]["dias_agenda"].discard(dia)
206                 container_alvo.bgcolor = "transparent"
207                 container_alvo.border = ft.Border(
208                     ft.BorderSide(1, "white30"), ft.BorderSide(1, "white30"),
209                     ft.BorderSide(1, "white30"), ft.BorderSide(1, "white30")
210                 )
211                 texto_alvo.color = "white"
212             else:
213                 automacoes[pino_rele]["dias_agenda"].add(dia)

```

```

205         container_alvo.bgcolor = "blue"
206         container_alvo.border = ft.Border(
207             ft.BorderSide(1, "blue"), ft.BorderSide(1, "blue"),
208             ft.BorderSide(1, "blue"), ft.BorderSide(1, "blue")
209         )
210         texto_alvo.color = "white"
211
212     salvar_dados_no_disco()
213     page.update()
214
215     dias_semana = ["Seg", "Ter", "Qua", "Qui", "Sex", "Sáb", "Dom"]
216     lista_botoes = []
217
218     for d in dias_semana:
219         esta_selecionado = d in automacoes[pino_rele]["dias_agenda"]
220         bgcolor = "blue" if esta_selecionado else "transparent"
221         cor_borda = "blue" if esta_selecionado else "white30"
222
223         container_chip = ft.Container(
224             content=ft.Text(d, size=11, weight="bold", color="white"),
225             alignment="center",
226             padding=8,
227             width=45,
228             height=32,
229             border_radius=6,
230             border=ft.Border(
231                 ft.BorderSide(1, cor_borda), ft.BorderSide(1, cor_borda),
232                 ft.BorderSide(1, cor_borda), ft.BorderSide(1, cor_borda)
233             ),
234             bgcolor=bgcolor,
235             data=d
236         )
237
238         detector_clique = ft.GestureDetector(
239             content=container_chip,
240             on_tap=alternar_dia_btn
241         )
242
243         buttons_dict[d] = container_chip
244         lista_botoes.append(detector_clique)
245
246     txt_hora.value = automacoes[pino_rele]["hora_agenda"]
247     if automacoes[pino_rele]["timer_minutos"] > 0:
248         txt_timer.value = str(automacoes[pino_rele]["timer_minutos"])
249
250     return ft.ExpansionTile(
251         title=ft.Text("Configurar Temporizador e Agenda", size=13, color="blue200"),
252         maintain_state=True,
253         controls=[
254             ft.Container(
255                 padding=10,
256                 bgcolor="white10", # Opacidade branca leve e segura sobre o fundo
257                 border_radius=8,
258                 content=ft.Column([
259                     ft.Row([
260                         ft.Icon(ft.Icons.TIMER, color="amber"),
261                         ft.Text("Desligar em:", weight="w500"),
262                         txt_timer,
263                         ft.IconButton(
264                             icon=ft.Icons.PLAY_ARROW_ROUNDED,
265                             icon_color="green",
266                             on_click=lambda e: acionar_temporizador(pino_rele,
267                                 txt_timer, switch_componente)
268                         )
269                     ], alignment="spaceBetween"),
270
271                     ft.Divider(height=10, color="white10"),
272
273                     ft.Row([
274                         ft.Icon(ft.Icons.SCHEDULE, color="blue"),
275                         ft.Text("Ligar às:", weight="w500"),
276                         txt_hora,

```

```

276         ft.IconButton(icon=ft.Icons.SAVE, icon_color="blue",
277                        on_click=salvar_agenda)
278     ], alignment="spaceBetween"),
279
280     ft.Text("Dias da semana:", size=12, color="white60"),
281     ft.Row(lista_botoes, wrap=True, alignment="center", spacing=5) #
282     Removido container com bug de tamanho fixo
283     ], spacing=10)
284 )
285
286
287 # --- MONTAGEM DOS CARDS ---
288 card_luz = ft.Card(
289     content=ft.Container(
290         padding=12,
291         content=ft.Column([
292             ft.Row([
293                 ft.Row([ft.Icon(ft.Icons.LIGHTBULB, color="amber", size=28),
294                        ft.Text("Luz (D6)", size=18, weight="w500")]),
295                 switch_luz
296             ], alignment="spaceBetween"),
297             criar_bloco_automacao("casa/rele1", txt_timer_d6, txt_hora_d6,
298                                  buttons_d6, switch_luz)
299         ])
300     )
301
302 card_tomada = ft.Card(
303     content=ft.Container(
304         padding=12,
305         content=ft.Column([
306             ft.Row([
307                 ft.Row([ft.Icon(ft.Icons.POWER, color="blueGrey200", size=28),
308                        ft.Text("Tomada (D7)", size=18, weight="w500")]),
309                 switch_tomada
310             ], alignment="spaceBetween"),
311             criar_bloco_automacao("casa/rele2", txt_timer_d7, txt_hora_d7,
312                                  buttons_d7, switch_tomada)
313         ])
314     )
315
316 # --- RENDERIZAÇÃO DA INTERFACE ---
317 page.add(
318     ft.Container(height=15),
319     ft.Row([status_led, status_text], alignment="center"),
320     ft.Divider(),
321     card_luz,
322     card_tomada
323 )
324 page.update()
325
326 # --- PROCESSAMENTO ASSÍNCRONO EM BACKGROUND THREAD ---
327 def conectar_mqtt_e_agendar():
328     try:
329         mqttc.connect(MQTT_SERVER, MQTT_PORT, 60)
330         mqttc.loop_start()
331     except Exception:
332         pass
333
334 def loop_verificacao_horario():
335     while True:
336         agora = datetime.datetime.now()
337         dias_map = ["Seg", "Ter", "Qua", "Qui", "Sex", "Sáb", "Dom"]
338         dia_atual_str = dias_map[agora.weekday()]
339         hora_atual_str = agora.strftime("%H:%M")
340
341         for pino, dados in automacoes.items():
342             if dados["hora_agenda"] == hora_atual_str and dia_atual_str in
343                 dados["dias_agenda"]:
344                 mqttc.publish(pino, "ON")

```

```
342         if pino == "casa/rele1":
343             switch_luz.value = True
344         if pino == "casa/rele2":
345             switch_tomada.value = True
346         page.update()
347
348         time.sleep(60)
349
350     threading.Thread(target=conectar_mqtt_e_agendar, daemon=True).start()
351     threading.Thread(target=loop_verificacao_horario, daemon=True).start()
352
353 ft.app(main)
```